

RunnableWithMessageHistory, Sessions & Multiple Conversations

1. Our Learning Journey

Friends, during the last four days we built conversation history step-by-step.

Day 1

Stateless Calls



Day 2

Manual Conversation History



Day 3

History Window



Day 4

Legacy Memory APIs



Day 5

Runnable-Based Message History

Today is very important because now we move from older APIs such as:

ConversationBufferMemory

ConversationChain

towards:

RunnableWithMessageHistory

2. Main Idea of RunnableWithMessageHistory

RunnableWithMessageHistory wraps another Runnable.

It can:

Read Conversation History



Execute Original Runnable



Store New Conversation

By default, a configurable:

session_id

is used to decide which conversation history should be used.

3. Day 5 Main Goal

By the end of this session, students should clearly understand:

What is Message History?



What is a Session?



Why Session ID?



InMemoryChatMessageHistory



RunnableWithMessageHistory



Same Session → Same History



Different Session → Different History

Today's main components are:

InMemoryChatMessageHistory

RunnableWithMessageHistory

session_id

4. Connect with Module 7 — Runnable

In Module 7 we already learned:

chain = prompt | model | parser

This creates a Runnable pipeline.

Now imagine:

Runnable

+

Conversation History

We want LangChain to manage history around that Runnable.

That is where:

RunnableWithMessageHistory

comes into the picture.

5. Simple Architecture

Normal Runnable



RunnableWithMessageHistory



History-Aware Runnable

6. Why Is It Called a Wrapper?

Suppose we already have:

llm

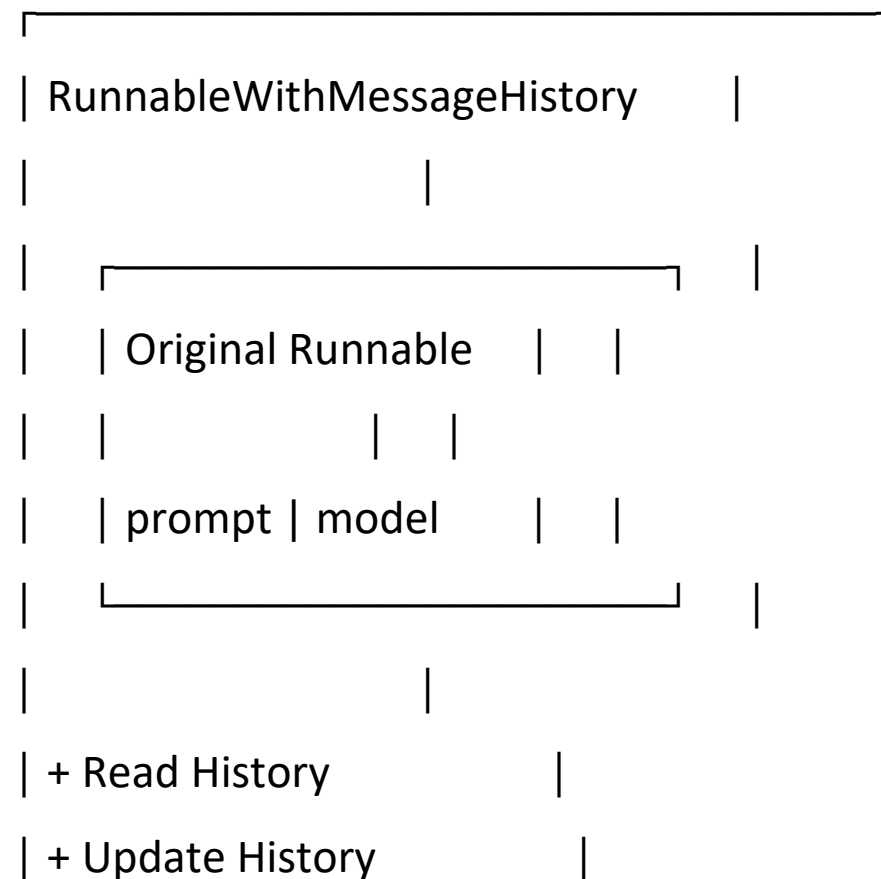
or:

chain = prompt | llm

We do not want to completely replace the original Runnable.

We want to add history management around it.

Conceptually:



|

So:

RunnableWithMessageHistory wraps another Runnable and adds message-history handling around it.

7. What Is Message History?

Conversation history normally contains:

HumanMessage

AIMessage

HumanMessage

AIMessage

...

Current LangChain provides chat-history abstractions for storing these message objects.

One simple implementation is:

InMemoryChatMessageHistory

It stores messages in application memory.

8. What Does "InMemory" Mean?

Very simple:

InMemory



Stored in Application Memory

It does **not automatically mean**:

Permanent Database

If the Python application stops, do not assume this in-memory history will remain permanently.

Persistent history is a separate topic.

9. Program 1 — Basic InMemoryChatMessageHistory

```
from langchain_core.chat_history import (  
    InMemoryChatMessageHistory  
)
```

```
# =====
```

```
# CREATE MESSAGE HISTORY
```

```
# =====
```

```
history = InMemoryChatMessageHistory()
```

```
# =====
```

```
# ADD USER MESSAGE
```

```
# =====
```

```
history.add_user_message(  
    "My name is Durga."  
)
```

```
# =====
```

```
# ADD AI MESSAGE
```

```
# =====
```

```
history.add_ai_message(  
    "Nice to meet you, Durga."  
)
```

```
# =====
```

```
# DISPLAY HISTORY
```

```
# =====
```

```
print("CONVERSATION HISTORY")
```

```
print("-----")
```

```
for message in history.messages:
```

```
    print(
        type(message).__name__,
        ":",
        message.content
    )
```

10. Conceptual Structure

InMemoryChatMessageHistory



[

HumanMessage,

AIMessage,

```
HumanMessage,  
AIMessage,  
...  
]
```

Very important:

InMemory



Application Memory

It does not automatically mean permanent storage.

11. Important Methods

To add a user message:

```
history.add_user_message(  
    "My name is Durga."  
)
```

To add an AI message:

```
history.add_ai_message(  
    "Nice to meet you."  
)
```

To access all messages:

history.messages

12. Why Do We Need Sessions?

Suppose two users are chatting.

Durga



Conversation 1

Ravi



Conversation 2

Should both users share the same history?

No

Each conversation should have its own history.

This leads us to:

Session

13. What Is a Session?

A session represents one separate conversation context.

Example:

Session: durga



Durga's Conversation History

Session: ravi



Ravi's Conversation History

14. Why Session ID?

A session_id tells the application:

Which conversation history should be used for this request?

Example:

session_id = "durga"



Use Durga History

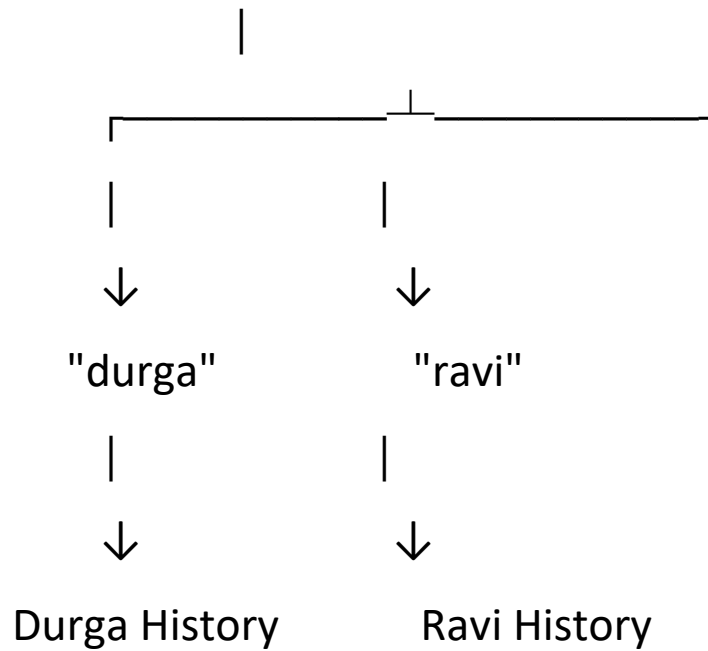
session_id = "ravi"



Use Ravi History

15. Session Memory Architecture

STORE



16. Using a Python Dictionary as Store

We can create:

```
store = {}
```

Conceptually:

```
store = {  
    "durga": DurgaHistory,  
    "ravi": RaviHistory  
}
```

Each session ID points to its own conversation history.

17. `get_session_history()`

We create a function:

```
def get_session_history(session_id):  
  
    if session_id not in store:  
  
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )  
  
    return store[session_id]
```

This function receives:

`session_id`

and returns the correct history.

18. How `get_session_history()` Works

Suppose:

`session_id = "durga"`

The function checks:

Is "durga" already in store?

If no:

Create New InMemoryChatMessageHistory



Store it under "durga"

If yes:

Return Existing Durga History

19. Flow of get_session_history()

session_id



Check Store



Does History Exist?

/ \

No Yes



Create Reuse

History Existing History

\ /



Return History

20. Wrap the Model with Message History

```
chatbot = RunnableWithMessageHistory(  
    llm,  
    get_session_history  
)
```

Concept:

llm



Wrapped by



RunnableWithMessageHistory



History-Aware Chatbot

21. Complete Basic Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
from langchain_core.chat_history import (  
    InMemoryChatMessageHistory  
)
```

```
from langchain_core.runnables.history import (  
    RunnableWithMessageHistory  
)
```

```
# =====  
# CREATE MODEL  
# =====
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# =====
```

```
# STORE
```

```
# =====
```

```
store = {}
```

```
# =====
```

```
# GET SESSION HISTORY
```

```
# =====
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )
```

```
    return store[session_id]
```

```
# =====  
# WRAP MODEL WITH MESSAGE HISTORY  
# =====
```

```
chatbot = RunnableWithMessageHistory(  
    llm,  
    get_session_history  
)
```

```
# =====  
# FIRST MESSAGE  
# =====
```

```
response = chatbot.invoke(  
  
    [  
        HumanMessage(  

```

```
        content="My name is Durga."
    )
],

config={
    "configurable": {
        "session_id": "durga"
    }
}
)

# =====
# SECOND MESSAGE
# =====

response = chatbot.invoke(

    [
        HumanMessage(
```

```
        content="What is my name?"
    )
],

config={
    "configurable": {
        "session_id": "ravi"
    }
}
)

print(response.content)
```

22. Important Observation

First call:

"My name is Durga."



session_id = "durga"

So this information is stored under:

durga history

Second call:

"What is my name?"



session_id = "ravi"

Now the application uses:

ravi history

not:

durga history

Therefore, Ravi's session does not automatically receive Durga's conversation.

23. Same Session vs Different Session

Same Session

Message 1

session_id = "durga"



Durga History

Message 2

session_id = "durga"



Same Durga History

So previous context is available.

Different Session

Message 1

session_id = "durga"



Durga History

Message 2

session_id = "ravi"



Ravi History

These histories are separate.

24. Easy Memory Rule

SAME SESSION ID



SAME HISTORY

DIFFERENT SESSION ID



DIFFERENT HISTORY

This is one of the most important points of this topic.

25. config={"configurable": ...}

When invoking the wrapped Runnable, we provide:

```
config={  
    "configurable": {  
        "session_id": "durga"  
    }  
}
```

This tells the wrapper which session history should be used.

26. Important Part

"session_id": "durga"

means:

Use History Belonging to "durga"

Changing it to:

"session_id": "ravi"

means:

Use Ravi's History

27. Complete Session Selection Flow

User Request



session_id



get_session_history(session_id)



Correct Chat History



RunnableWithMessageHistory



Model



AI Response



Update Same Session History

28. Program — Multiple Conversations

Now let us maintain two different conversations:

Durga Session

Ravi Session

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
from langchain_core.chat_history import (
    InMemoryChatMessageHistory
)
```

```
from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)
```

```
# =====
```

```
# CREATE MODEL
```

```
# =====
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# =====
```

```
# STORE
```

```
# =====
```

```
store = {}
```

```
# =====
```

```
# GET SESSION HISTORY
```

```
# =====
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )
```

```
    return store[session_id]
```

```
# =====
```

```
# WRAP MODEL WITH MESSAGE HISTORY
```

```
# =====
```

```
chatbot = RunnableWithMessageHistory(  
    llm,
```

```
    get_session_history
```

```
)
```

```
# =====
```

```
# DURGA — MESSAGE 1
```

```
# =====
```

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="My name is Durga."
```

```
    )
```

```
],
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "durga"
```

```
    }
```

```
}
```

```
)
```

```
# =====  
# DURGA — MESSAGE 2  
# =====  
  
response = chatbot.invoke(  
  
    [  
        HumanMessage(  
            content="I am teaching Python."  
        )  
    ],  
  
    config={  
        "configurable": {  
            "session_id": "durga"  
        }  
    }  
)
```

```
# =====
```

```
# DURGA — MESSAGE 3
```

```
# =====
```

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="I am from Hyderabad."
```

```
    )
```

```
],
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "durga"
```

```
    }
```

```
}
```

```
)
```

```
# =====
```

```
# RAVI — MESSAGE 1
```

```
# =====
```

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="I am Ravi."
```

```
    )
```

```
],
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "ravi"
```

```
    }
```

```
}
```

```
)
```

```
# =====
```

```
# RAVI — MESSAGE 2
```

```
# =====
```

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="I like movies, especially Sunny Leone movies."
```

```
    )
```

```
],
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "ravi"
```

```
    }
```

```
}
```

```
)
```

```
# =====
```

```
# RAVI — MESSAGE 3
```

```
# =====
```

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="I am from Bangalore."
```

```
    )
```

```
],
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "ravi"
```

```
    }
```

```
}
```

```
)
```

```
# =====
```

```
# ASK ABOUT RAVI
```

```
# =====
```

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="What do you know about me?"
```

```
    )
```

```
],
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "ravi"
```

```
    }
```

```
}
```

```
)
```

```
print(response.content)
```

29. What Is Stored for Durga?

Conceptually:

```
store["durga"]
```



Human: My name is Durga.

AI: ...

Human: I am teaching Python.

AI: ...

Human: I am from Hyderabad.

AI: ...

30. What Is Stored for Ravi?

Conceptually:

```
store["ravi"]
```



Human: I am Ravi.

AI: ...

Human: I like movies...

AI: ...

Human: I am from Bangalore.

AI: ...

These are independent conversation histories.

31. Store Architecture

store

|

| — "durga"

| ↓

| InMemoryChatMessageHistory

| ↓

| Durga's Messages

|

| — "ravi"



InMemoryChatMessageHistory



Ravi's Messages

32. What Happens When Ravi Asks:

What do you know about me?

and we use:

"session_id": "ravi"

the wrapper selects:

Ravi History

So the model receives Ravi's previous conversation context.

It does not automatically receive Durga's history.

33. What If We Change the Session ID?

Suppose Ravi asks:

What do you know about me?

but we provide:

"session_id": "durga"

Then the application selects:

Durga History

So the conversation context changes.

Very important:

Same Question

+

Different Session ID

↓

Different History

↓

Potentially Different Answer

34. Main Role of Session ID

Think of session_id as a conversation identifier.

session_id

↓

Find Correct Conversation

↓

Load Correct History

Example:

"durga"



Durga Conversation

"ravi"



Ravi Conversation

35. RunnableWithMessageHistory — Core Responsibilities

Conceptually, the wrapper handles:

1. Identify Session



2. Get Session History



3. Add History Around Runnable Execution



4. Execute Runnable



5. Add New Messages Back to History

36. Compare with Manual History

Earlier we manually wrote:

```
messages.append(  
    HumanMessage(...)  
)
```

```
response = llm.invoke(  
    messages  
)
```

```
messages.append(  
    response  
)
```

Now:

`RunnableWithMessageHistory`

helps manage the history around the Runnable.

37. Manual vs `RunnableWithMessageHistory`

Manual Approach

Application



Append Human



Send History



Get AI



Append AI

RunnableWithMessageHistory

Session ID



Get History



Wrapped Runnable



Execute



Update History

38. Compare with Legacy Memory

Legacy

ConversationBufferMemory

+

ConversationChain

Runnable-Based Approach

InMemoryChatMessageHistory

+

RunnableWithMessageHistory

+

session_id

39. Easy Evolution Diagram

MANUAL

```
messages = []
```

```
messages.append(...)
```

↓

LEGACY

ConversationBufferMemory

ConversationChain

↓

RUNNABLE-BASED HISTORY

InMemoryChatMessageHistory

RunnableWithMessageHistory

session_id

40. Why Session Management Is Important

Real applications may have many conversations.

Example:

User 1

User 2

User 3

User 4

...

Each user or conversation needs isolated history.

Without session separation:

All Conversations



May Get Mixed

With session IDs:

User A → Session A → History A

User B → Session B → History B

User C → Session C → History C

41. InMemoryChatMessageHistory Is Still In-Memory

Even though we now support different sessions:

durga

ravi

suresh

our example still uses:

```
store = {}
```

and:

```
InMemoryChatMessageHistory
```

So this example is still based on application memory.

Do not assume permanent persistence.

42. What Happens When Program Stops?

Conceptually:

Python Program Running



store exists



Session histories available

But when the process stops, do not assume this in-memory dictionary remains permanently.

Persistent storage is a separate topic.

43. Three Main Concepts

1. Message History

HumanMessage

AIMessage

HumanMessage

AIMessage

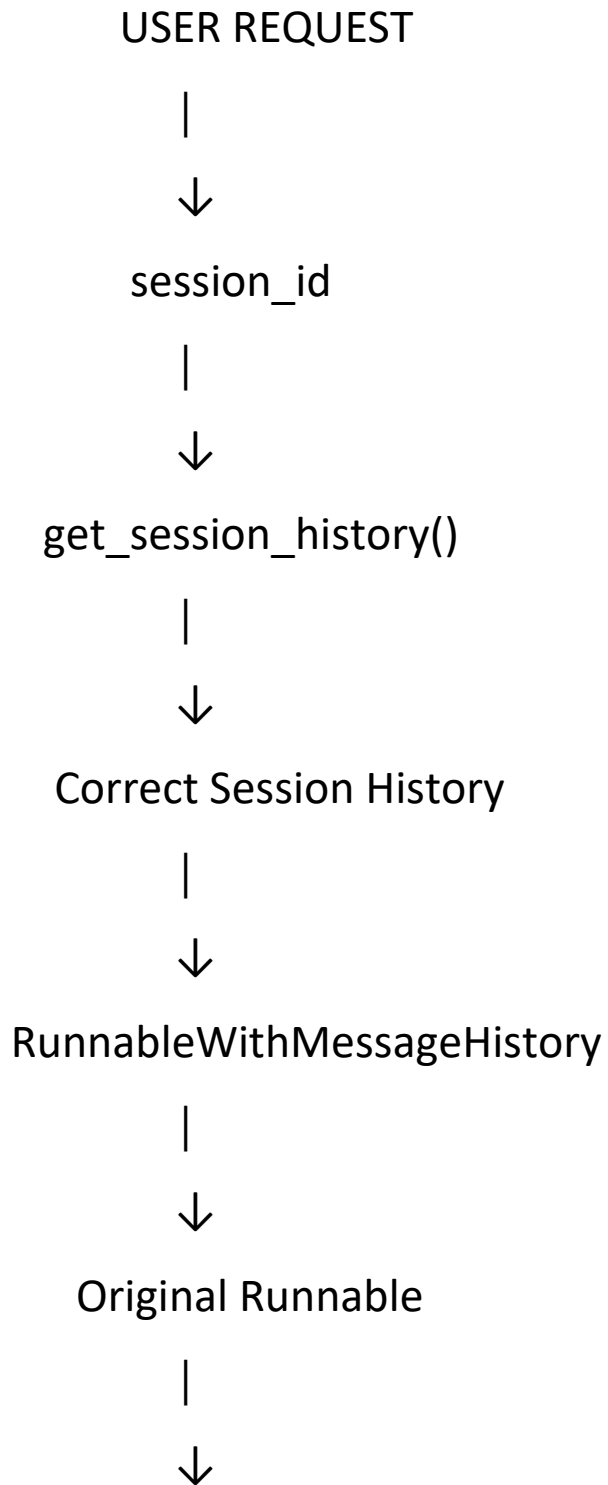
2. Session

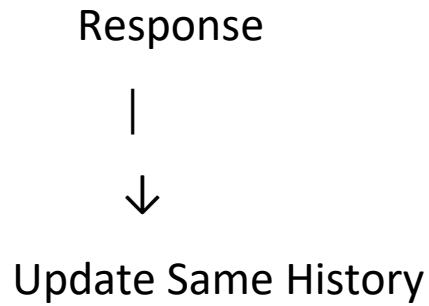
One Separate Conversation

3. Session ID

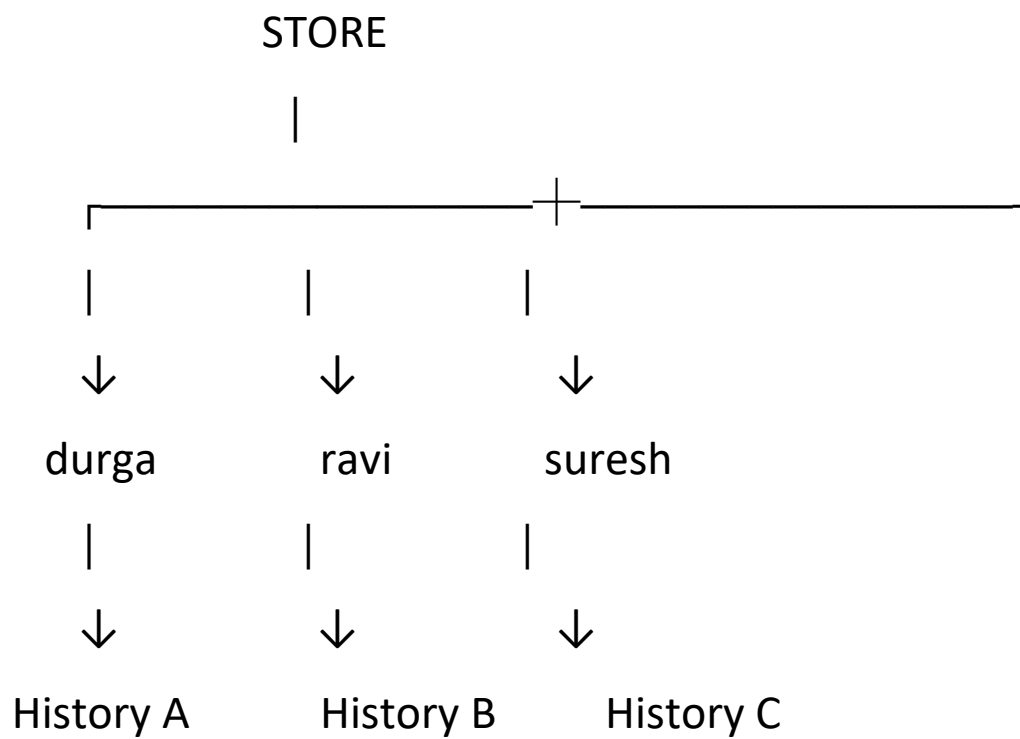
Identifier Used to Find
the Correct History

44. Master Architecture





45. Multiple Session Architecture



46. Most Important Rule

Same session_id



Same Conversation History

Different session_id



Different Conversation History

47. Important Key Points

1. InMemoryChatMessageHistory stores chat messages in application memory.
2. It stores message objects such as HumanMessage and AIMessage.
3. RunnableWithMessageHistory wraps another Runnable.
4. The wrapper adds conversation-history handling around the original Runnable.
5. session_id decides which conversation history should be used.
6. Same session ID means the same conversation history is reused.
7. Different session IDs maintain separate conversations.
8. A Python dictionary can be used as a simple session store.
9. get_session_history(session_id) returns the history belonging to a session.
10. If a session does not exist, we can create a new InMemoryChatMessageHistory.

11. InMemoryChatMessageHistory is not automatically permanent storage.
 12. Session management prevents histories of different conversations from getting mixed.
-

48. Common Mistake — Same Model Means Same History

Wrong:

Same Llm Object



Same Conversation

No.

Conversation history is selected using the session.

Correct thinking:

Same Model

+

Different session_id



Different History

49. Common Mistake — Different Sessions Share History

Wrong.

```
session_id = "durga"
```

and:

```
session_id = "ravi"
```

represent different histories in this design.

50. Common Mistake — InMemory Means Permanent

Wrong.

InMemoryChatMessageHistory

means history is stored in the application's memory.

It does not automatically mean permanent database storage.

51. Common Mistake — Forgetting Session ID

When using this pattern, the wrapper needs to know which history to use.

That is why we provide:

```
config={
    "configurable": {
        "session_id": "durga"
    }
}
```

52. MCQ 1

What is the main purpose of session_id?

- A. Select the LLM model
- B. Identify the correct conversation history
- C. Generate embeddings
- D. Set temperature

Answer: B

Conclusion

session_id identifies which conversation history belongs to the current conversation.

53. MCQ 2

What happens if two calls use the same session_id?

- A. They automatically use different histories
- B. They use the same session history
- C. The model is retrained
- D. The API key changes

Answer: B

Conclusion

Same session ID means the same conversation context can continue.

54. MCQ 3

What happens if two calls use different session IDs?

- A. They use separate conversation histories
- B. They always share history
- C. Both histories are deleted
- D. They must use different models

Answer: A

Conclusion

Different session IDs allow multiple independent conversations.

55. MCQ 4

What is InMemoryChatMessageHistory mainly used for?

- A. Store chat messages in application memory
- B. Generate vectors
- C. Store API keys
- D. Parse JSON

Answer: A

Conclusion

It provides a simple in-memory implementation for storing conversation messages.

56. MCQ 5

Why is RunnableWithMessageHistory called a wrapper?

- A. It replaces Python
- B. It adds history management around another Runnable
- C. It creates API keys
- D. It permanently trains the model

Answer: B

Conclusion

It wraps an existing Runnable and adds message-history behaviour around it.

57. Final Comparison

Concept	Purpose
InMemoryChatMessageHistory	Stores chat messages
store	Holds histories for different sessions
get_session_history()	Finds or creates the correct history
session_id	Identifies the conversation
RunnableWithMessageHistory	Adds history management around a Runnable

llm

Generates AI response

58. Final Memory Formula

MESSAGE HISTORY

+

SESSION ID

+

RUNNABLE

↓

HISTORY-AWARE CONVERSATION

And remember:

SAME SESSION

↓

SAME HISTORY

DIFFERENT SESSION

↓

DIFFERENT HISTORY

59. One-Line Conclusion

RunnableWithMessageHistory wraps a **Runnable** with conversation-history handling, while **session_id** identifies which conversation history should be used, allowing multiple independent conversations to be maintained separately.